

고급 인공지능

- 데이터의 변용: 벡터 DB와 랭체인

동양미래대학교

컴퓨터소프트웨어공학과

송원문

mooniesong@gmail.com

강의 개요

■ 주제

- 데이터의 변용: 벡터 DB와 랭체인

■ 내용

- 벡터 DB 원리, 아나콘다 환경 설정 및 API 키 발급
- 교재 3.3 ~ 4.2

■ 학습 목표

- 비정형 데이터를 벡터로 변환하는 임베딩의 개념을 익힌다.
- 아나콘다 및 API 환경 설정을 완수하고 첫 Hello World를 출력한다.

■ RAG 구현시 필요한 것

- 데이터
- 벡터 데이터베이스
- 프레임워크 – 랭체인

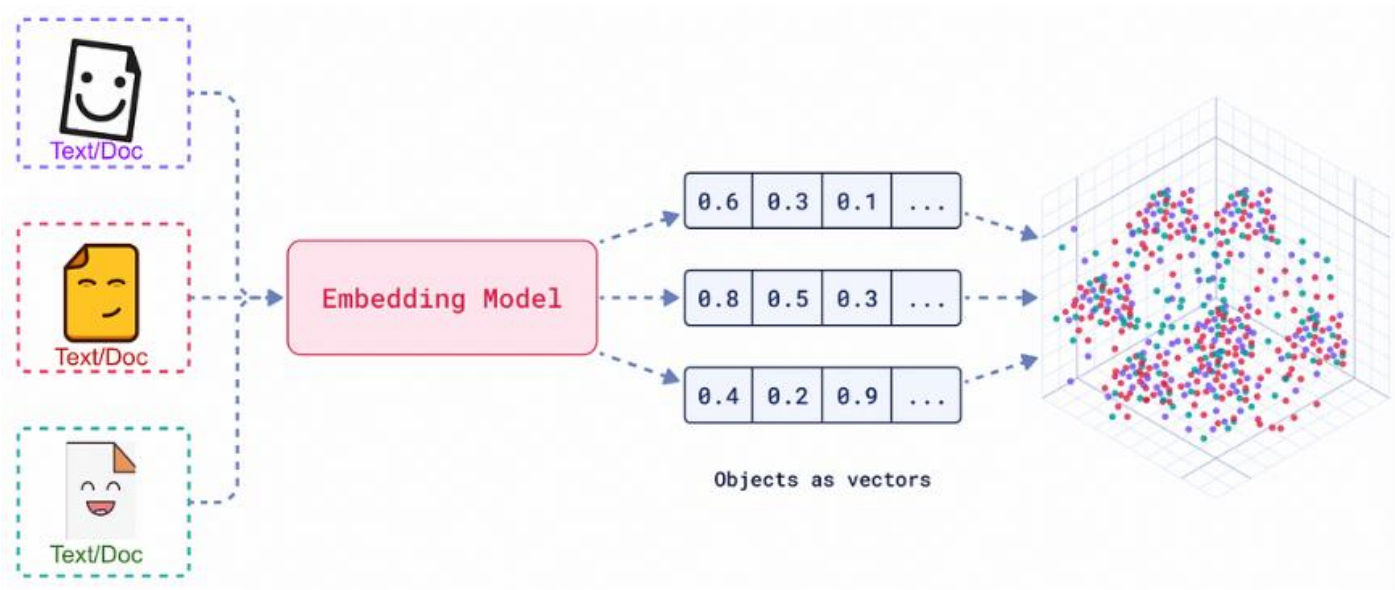
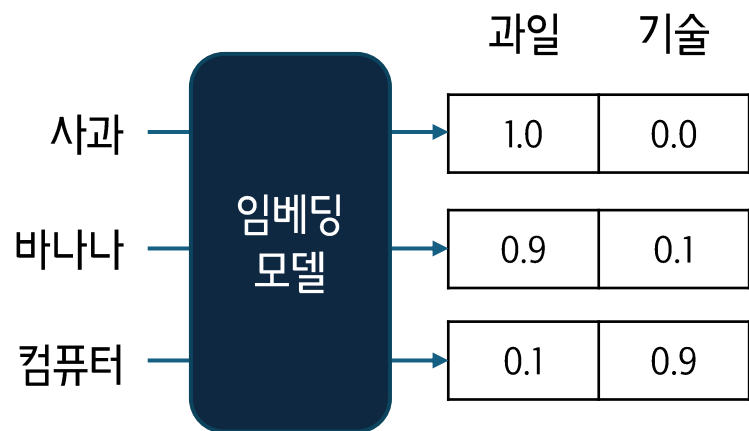
■ 랭체인 훑어보기

■ 랭체인을 사용하기 위한 환경 구성

- 아나콘다 환경 구성
- 라이브러리 설치
- API 키 발급

RAG 구현시 필요한 것 - 데이터

- 사용할 수 있는 데이터 형식에 특별히 제약은 없음. CSV, JSON, PDF 등의 파일이나 데이터베이스도 가능
- 중요한 것은 규범과 규제 - 개인정보, 저작권, 데이터 소유권, 활용권 등
- 임베딩(embedding) 데이터로부터 검색을 하기 위해서는 데이터를 벡터(vector)로 임베딩 해야 함
 - 복잡한 형식의 데이터를 컴퓨터가 이해하고 계산에 사용할 수 있도록 벡터(숫자의 리스트)로 바꾸는 것
 - 벡터의 비교 (벡터 공간상에서의 거리나 각도)를 통해서 유사한 단어인지 아닌지에 대해 이해(계산)할 수 있음



RAG 구현시 필요한 것 - 데이터

■ 임베딩 모델

- 벡터로의 임베딩을 위해서는 타당한 변환(계산) 알고리즘이 있어야 함
- Word2Vec, GloVe와 같은 벡터 임베딩 전용 라이브러리나 OpenAI 등 LLM 서비스에서 제공하는 API를 이용

■ Word2Vec을 이용한 임베딩 및 활용 실습

- 간단한 실습
- 네이버 영화 리뷰 데이터를 이용한 실습
 - JDK 설치 필요: <https://adoptium.net/>

■ GloVe를 이용한 임베딩 및 활용 실습

- 기존에 학습된 결과를 이용한 실습 (영어)

■ OpenAI API를 이용한 임베딩 실습

RAG 구현시 필요한 것 - 데이터

■ Word2Vec vs. GloVe

	Word2Vec	GloVe
핵심 아이디어	"주변 단어를 맞혀라"	"단어 쌍의 동시 등장 확률 비를 맞춰라"
방법	예측 기반(Prediction-based). 윈도우(Window) 내의 주변 단어를 통해 중심 단어를 예측(CBOW)하거나, 중심 단어로 주변 단어를 예측(Skip-gram)하며 가중치를 업데이트	카운트 기반 + 예측 기반의 혼합(Global Vectors). 전체 코퍼스에서 단어 i와 단어 j가 함께 나타난 횟수를 행렬로 만든 뒤, 그 동시 등장 확률(Co-occurrence Probability)의 비율을 예측하도록 임베딩
알고리즘 유형	예측 기반 (Prediction)	카운트 기반 (Count-based) + 행렬 분해
학습 단위	윈도우 기반 (국소적)	전체 코퍼스 (전역적)
장점	단어 간 유추(Analogy)에 뛰어남 (예: King - Man + Woman = Queen)	전체 통계 정보를 잘 반영하여 안정적임 드물게 등장하는 단어에 대해서도 좋은 경우가 많음
단점	전체적인 통계 정보를 무시함	큰 행렬을 만들어야 해서 메모리 소모가 큼
주요 용도	단어간 세밀한 관계 파악	주제 분류 및 전체 맥락 파악

RAG 구현시 필요한 것 - 데이터

■ 벡터 공간에서 단어간 유추의 개념; $\text{King} - \text{Man} + \text{Woman} = \text{Queen}$

- 의미의 기하학적 구조

Word2Vec은 단어를 수백 차원의 공간에 점으로 표현

이때 '성별'과 같은 추상적인 개념은 공간상에서 특정 '방향'과 '길이', 즉, 벡터로 존재

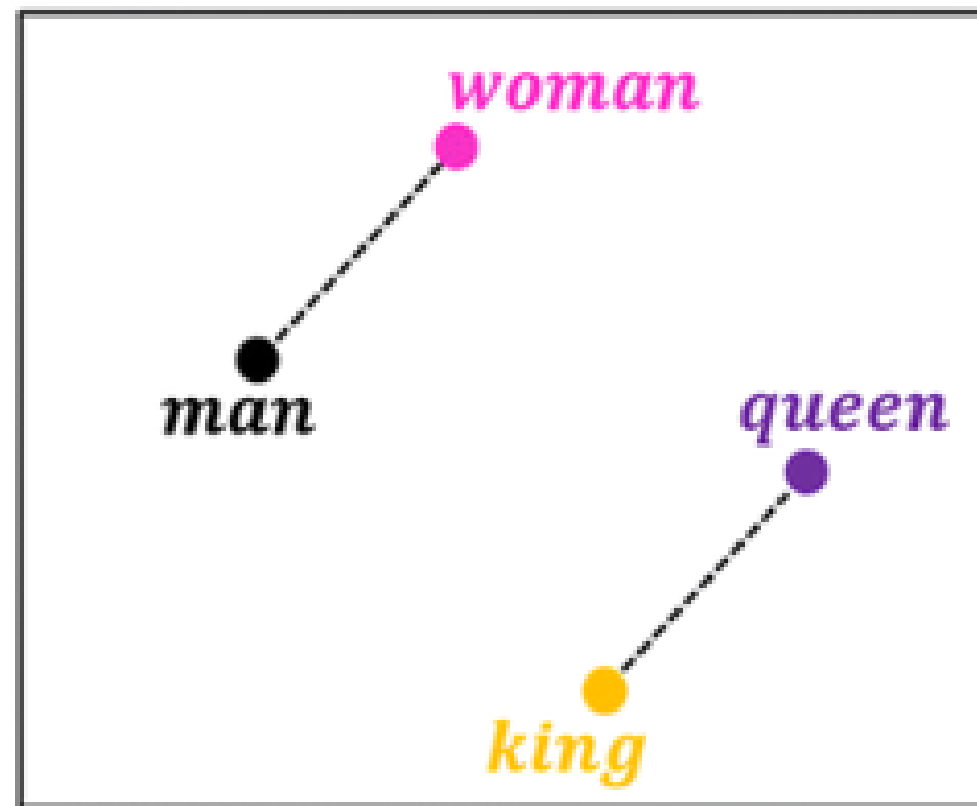
- 평행 이동

Man에서 Woman으로 이동하는 벡터와

King에서 Queen으로 이동하는 벡터는 거의 평행할 것

따라서 King 좌표에 (Woman - Man)의 차이 만큼을 더해주면

Queen 좌표와 매우 가까운 지점에 도달하게 되는 원리



■ RAG 구현시 필요한 것

- 데이터
- 벡터 데이터베이스
- 프레임워크 – 랭체인

■ 랭체인 훑어보기

■ 랭체인을 사용하기 위한 환경 구성

- 아나콘다 환경 구성
- 라이브러리 설치
- API 키 발급

RAG 구현시 필요한 것 – 벡터 데이터베이스

- 단어나 문장의 유사도는, 수많은 차원의 벡터로 변환된 값을 이용하여 계산하게 됨.
- 매번 변환할 수 없고, 벡터로 변환된 값을 벡터 처리에 특화된 데이터베이스에 저장하는 것이 유리
- 벡터 데이터베이스 = 벡터 데이터에 특화된 관리/검색(유사도 기반) 등을 제공하는 DB 시스템

	일반 데이터베이스 / SQL	벡터 데이터베이스
핵심 아이디어	키워드가 정확히 포함되어 있는가?	의미적으로 얼마나 가까운가?
방식 및 특징	쿼리에 대한 검색 방식: 문자열 매칭 (LIKE 연산 등) 특징: '강아지'라고 검색하면 '개', '퍼피', '반려견'이라는 단어가 들어간 문장은 찾지 못함, 정확한 결과 검색	쿼리에 대한 검색 방식: 유사도/거리 계산 특징: '강아지'를 벡터로 변환 하고, 가장 유사한 벡터 데이터를 검색, 정확하지 않아도 유사한 결과를 검색
쿼리 예시	SELECT * FROM documents WHERE content LIKE '%강아지%';	query_text = "강아지" query_vector = embedding_model.encode(query_text) results = vector_db.search(query_vector, top_k=3)
결과 예시	(O) "우리집 <u>강아지</u> 는 귀여워요." (X) "내 <u>반려견</u> 이름은 초코야." (X) "귀여운 <u>골든 리트리버</u> 분양합니다."	"우리집 <u>강아지</u> 는 귀여워요." (유사도 0.98) "내 <u>반려견</u> 이름은 초코야." (유사도 0.92) "귀여운 <u>골든 리트리버</u> 분양합니다." (유사도 0.88)

RAG 구현시 필요한 것 – 벡터 데이터베이스

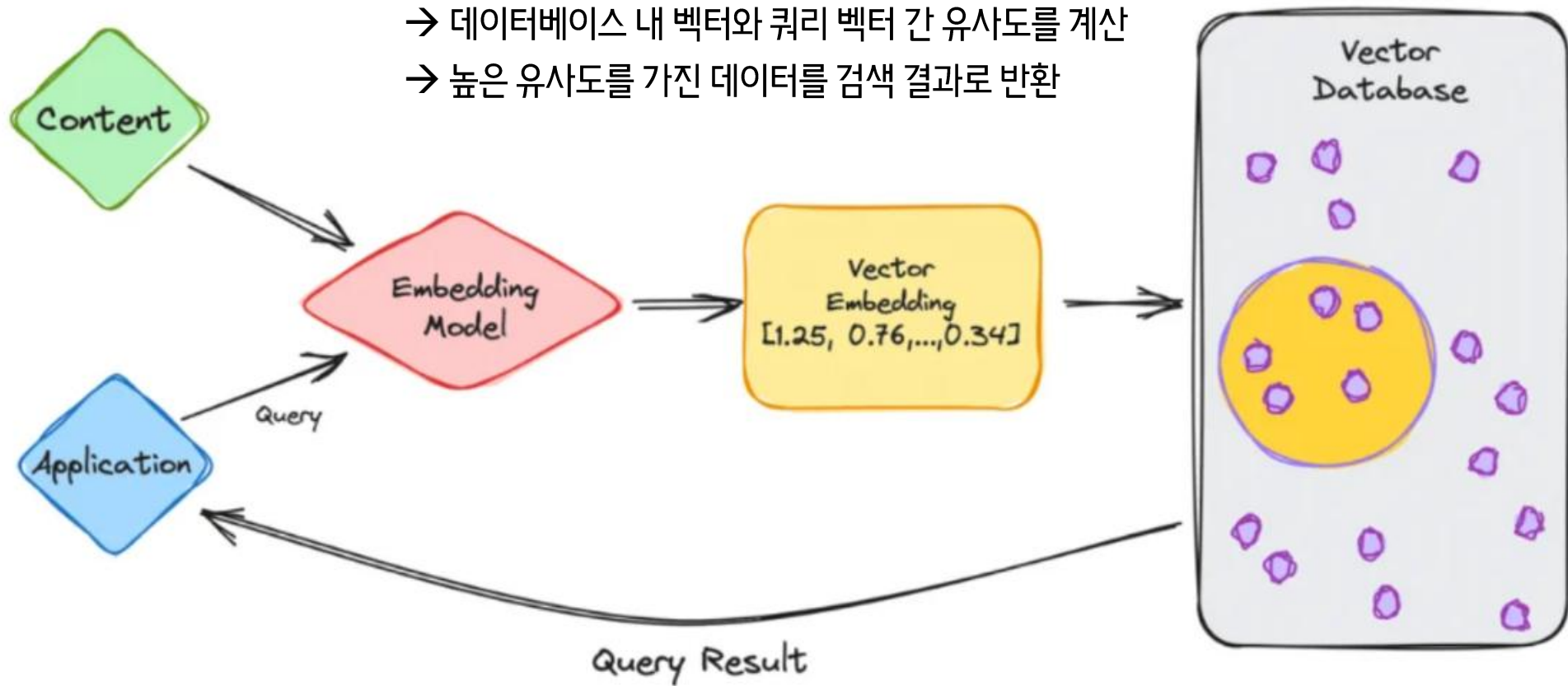
[벡터 데이터베이스 구성] 데이터를 벡터 형태로 인코딩

→ 인코딩된 벡터를 벡터 데이터베이스에 저장

[데이터 검색] 사용자 쿼리를 벡터로 인코딩

→ 데이터베이스 내 벡터와 쿼리 벡터 간 유사도를 계산

→ 높은 유사도를 가진 데이터를 검색 결과로 반환



RAG 구현시 필요한 것 – 벡터 데이터베이스

■ 주요 벡터데이터베이스 기술 비교

	Pinecone	Milvus	Qdrant	Chroma	Elasticsearch	FAISS
개발사	Pinecone (미국)	Zilliz (중국/LF AI)	Qdrant (독일)	Chroma (미국)	Elastic (네덜란드/미국)	Meta (Facebook)
출시 시기	2021년	2019년 (오픈소스)	2021년	2023년	2010년 (벡터는 2022년)	2017년
유형	SaaS (완전관리형)	Cloud-Native DB	High-Perf DB	AI-Native DB	Search Engine	Library (라이브러리)
주요 언어	Go / Python (API)	Go / Python / C++	Rust	Python / JavaScript	Java	C++ / Python
설치 방식	필요 없음 (클라우드)	Docker / K8s	Docker / Binary	pip install	서버 설치 / 클라우드	pip install
장점	인프라 관리 불필요, 빠른 도입	압도적 확장성, 대형 엔터프라이즈 최적화	Rust 기반의 고성능, 효율적 필터링	매우 쉬운 사용법, LLM 연동 최적화	기존 텍스트 검색과 결합(하이브리드)	극한의 검색 속도, GPU 가속 지원
단점	높은 비용, 데이터 외부 저장(보안)	운영 난이도 높음, 자원 소모 큼	인지도 및 생태계 성장 중	대규모 분산 처리 미흡	벡터 전용보다 느림, 메모리 점유 높음	DB 기능(저장/관리) 부족, 직접 개발 필요
추천 대상	스타트업, 빠른 MVP 제작	대규모 트래픽 기업	고성능/저비용 서버 운영	초보자, 개인 프로젝트	기존 Elastic 사용자	연구용, 알고리즘 구현

■ RAG 구현시 필요한 것

- 데이터
- 벡터 데이터베이스
- 프레임워크 – 랭체인

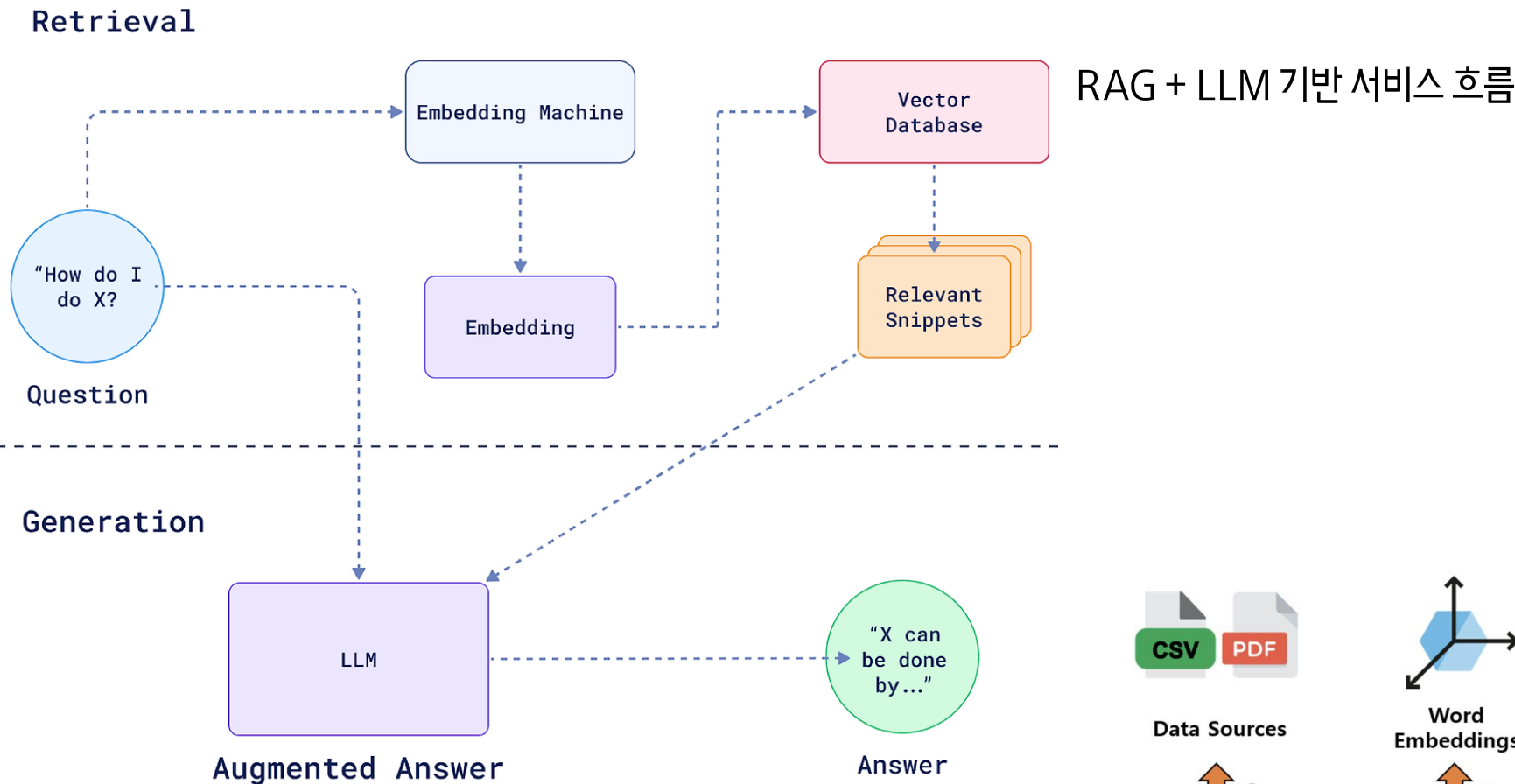
■ 랭체인 훑어보기

■ 랭체인을 사용하기 위한 환경 구성

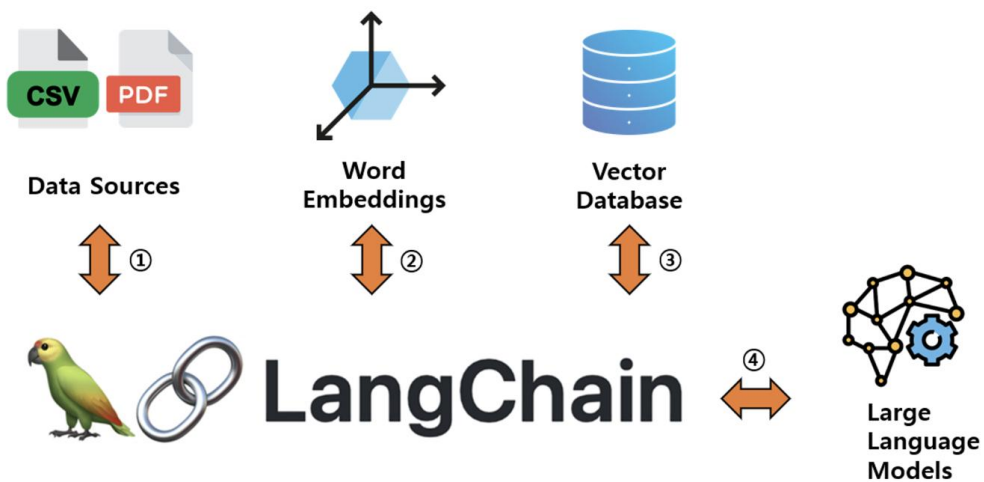
- 아나콘다 환경 구성
- 라이브러리 설치
- API 키 발급

RAG 구현시 필요한 것 – 프레임워크: 랭체인

■ 랭체인 (LangChain) – LLM 기반 서비스 개발(흐름 제어)을 위한 모듈 및 프레임워크 기술



랭체인 개념



■ RAG 구현시 필요한 것

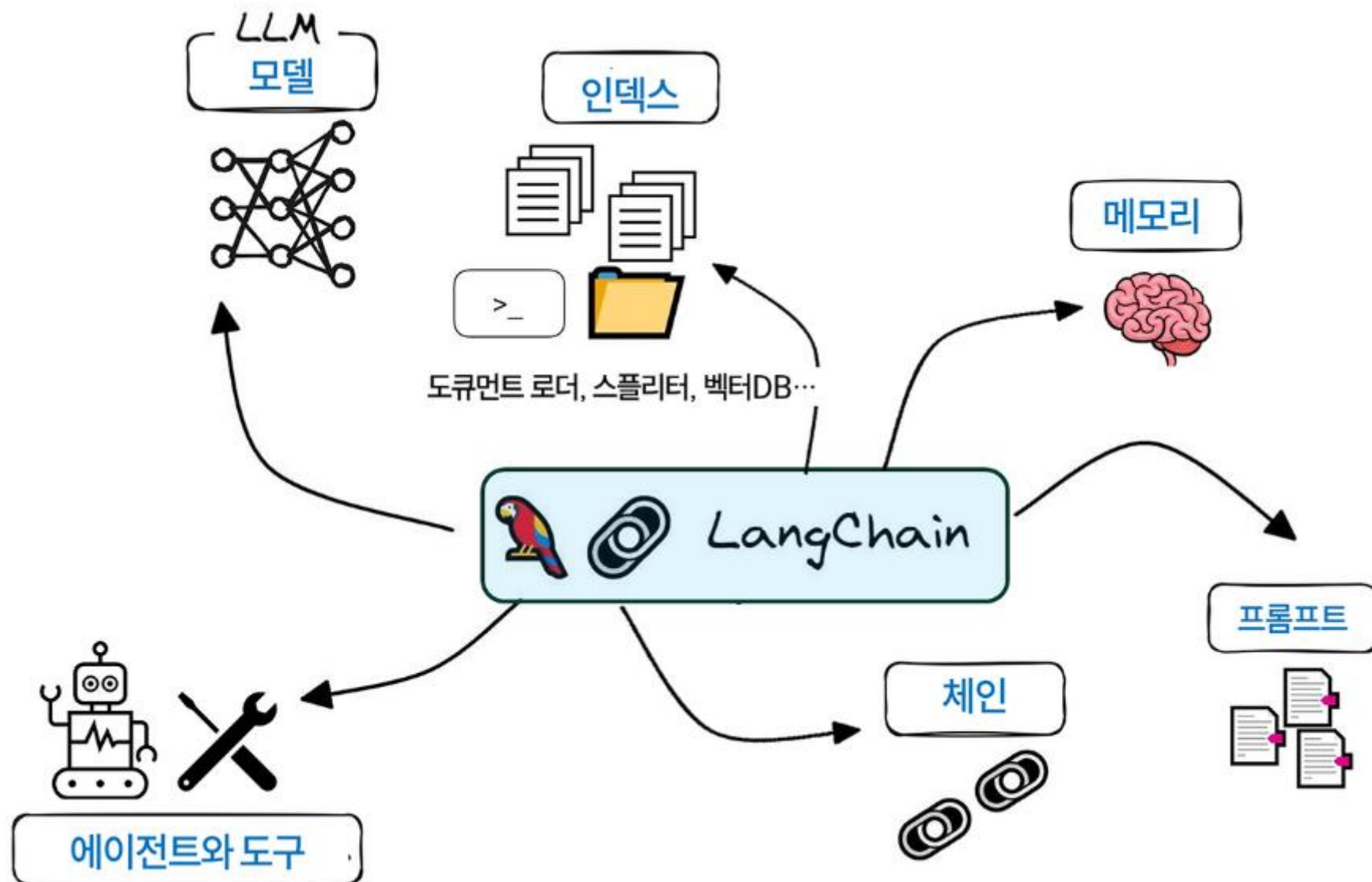
- 데이터
- 벡터 데이터베이스
- 프레임워크 – 랭체인

■ 랭체인 훑어보기

■ 랭체인을 사용하기 위한 환경 구성

- 아나콘다 환경 구성
- 라이브러리 설치
- API 키 발급

랭체인 훑어보기



랭체인 훑어보기

■ 랭체인 (LangChain)

- 2022년 말, 해리슨 체이스에 의해 오픈소스로 시작, 커뮤니티에 의해 지속 발전
- 앵무새=언어모델, 체인=언어모델과 그것을 활용할 수 있는 도구를 결합
- RAG 구현시 필요한 것들 = 임베딩, 유사도 검색, 랭킹 처리
- LLM 활용을 위해 필요한 것들 = 프롬프트 처리, LLM 연결
- 그 외 다양한 도구들과 LLM을 연결



■ RAG 구현시 필요한 것

- 데이터
- 벡터 데이터베이스
- 프레임워크 – 랭체인

■ 랭체인 훑어보기

■ 랭체인을 사용하기 위한 환경 구성

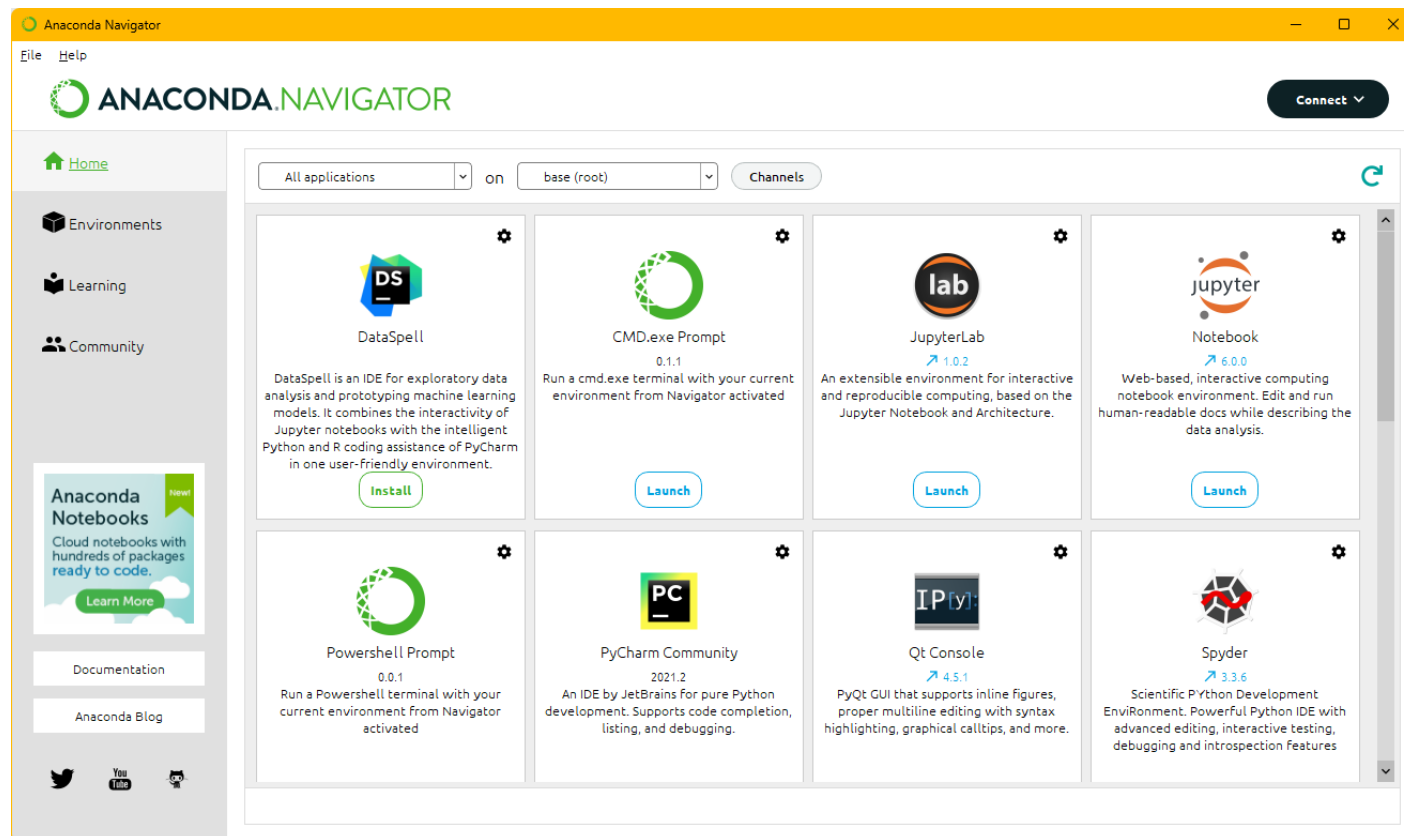
- 아나콘다 환경 구성
- 라이브러리 설치
- API 키 발급

랭체인 사용 환경 구성 - 아나콘다

■ 파이썬 프로그래밍을 위해서는 다양한 환경 설정이 필요. '아나콘다(Anaconda)'는 환경 설정을 도와주는 패키지 도구

- 주요 제공 기능

- 파이썬(Python) 엔진
- 가상환경 관리자 (Conda): 프로젝트마다 서로 다른 파이썬 버전이나 라이브러리를 독립적으로 쓸 수 있게 나누어 주는 역할
- 다양한 사전 설치 라이브러리: 데이터 분석(Pandas), 수치 계산(NumPy), 시각화(Matplotlib), 머신러닝(Scikit-learn) 등 기본 포함



■ 아나콘다의 특징

- 복잡한 설치 과정의 생략 (Dependency 해결)
 - 파이썬 라이브러리들은 서로 의존 관계가 복잡(A는 B필요, B는 특정 버전 C 필요 등). 이러한 부분을 미리 정리해서 설치 제공
- 가상환경(Virtual Environment) 분리
 - [A 프로젝트] 파이썬 3.8 + 텐서플로우 1.15 사용, [B 프로젝트] 파이썬 3.12 + 텐서플로우 2.15 사용
 - 한 컴퓨터에서 다양한 버전의 프로그램/라이브러리를 사용하는 경우 충돌 발생 가능. 가상환경으로 이러한 충돌을 방지하고 개별적으로 운용
- GUI 기반의 편리함 (Anaconda Navigator)
 - 마우스 클릭만으로 라이브러리를 관리하고 주피터 노트북(Jupyter Notebook)과 같은 사용자 친화적 툴을 실행할 수 있는 화면을 제공

랭체인 사용 환경 구성 - 아나콘다

■ 파이선 개발을 위한 다양한 도구

- PyCharm, Visual Studio Code



제공사	JetBrains	Microsoft
성격	IDE (통합 개발 환경)	Text Editor (확장 가능한 편집기)
설치 및 설정	설치 즉시 모든 도구 사용 가능 (무거움)	필요한 기능을 플러그인으로 직접 설치 (가벼움)
비용	유료(Professional) / 무료(Community)	완전 무료 (오픈소스 기반)
메모리 점유	높음 (대형 프로젝트에 유리)	낮음 (가볍고 빠름)
코드 분석	매우 강력 (자동 완성, 리팩토링 최적화)	우수 (Pylance 확장 프로그램 설치 시)
가상환경 관리	자동 감지 및 설정 UI가 매우 직관적	설정 파일(json)이나 명령어로 직접 지정

■ RAG 구현시 필요한 것

- 데이터
- 벡터 데이터베이스
- 프레임워크 – 랭체인

■ 랭체인 훑어보기

■ 랭체인을 사용하기 위한 환경 구성

- 아나콘다 환경 구성
- 라이브러리 설치
- API 키 발급

■ 설치할 라이브러리

- langchain
- openai
 - OpenAI에서 제공하는 모델의 api를 호출하기 위한 라이브러리



설립 배경	• 2015년에 Elon Musk, Sam Altman 등의 주도로, 인공지능 기술의 발전과 안전한 AI 개발을 위해 설립
주요 활동	• 인공지능 모델 및 알고리즘 개발, 인공지능 기술의 안전성과 윤리성 연구, 인공지능 기술의 대중화와 활용 촉진
주요 제품	• GPT (Generative Pre-trained Transformer; 자연어 처리 모델) – “ChatGPT” • DALL-E(텍스트를 입력받아 이미지를 생성하는 모델), Whisper(음성 인식 및 변환 모델), Codex(코드 생성 모델)
지향점 (개방성/투명성)	• 대부분의 연구 성과를 오픈소스로 공개하며, 연구 과정과 결과에 대한 투명성을 강조 • 이를 통해 인공지능 기술의 발전과 윤리적 활용을 도모

■ 설치할 라이브러리

- huggingface-hub
 - 전세계 개발자들이 만든 인공지능 모델과 데이터를 자유롭게 공유하는 AI 특화된 깃허브
 - 자연어처리와 관련, 여러 LLM들과 이를 사용할 수 있는 api 및 관련 라이브러리를 제공



Hugging Face

모델(Models) [AI 모델 도서관]

- 다양한 AI 모델들의 공유 (언어 번역, 이미지 생성, 음성 인식 등)
- 이미 만들어진 고성능 모델을 가져와서 자유롭게 사용 가능

데이터셋(Datasets) [AI 공부용 교과서]

- AI를 학습시키는 데 필요한 다양한 데이터 공유
- 한국어 뉴스, 이미지 모음, 목소리 파일 등

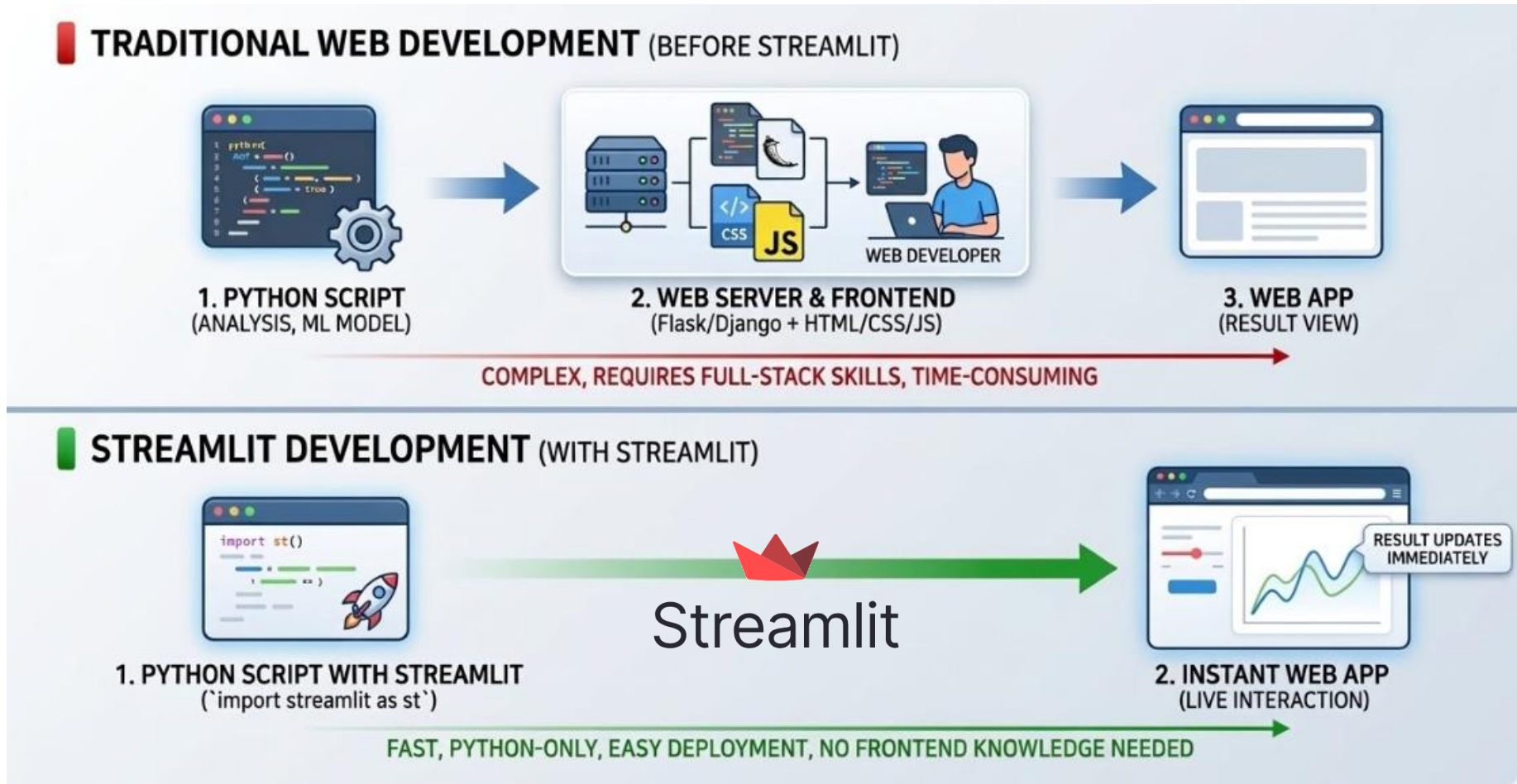
스페이스(Spaces) [AI 체험 전시장]

- 복잡한 코딩 없이도 AI 모델의 작동을 웹에서 바로 클릭해 보며 테스트

랭체인 사용 환경 구성 – 라이브러리

■ 설치할 라이브러리

- streamlit
 - 파이썬을 이용하여 웹 어플리케이션을 빠르고 쉽게 개발할 수 있도록 기능을 제공하는 라이브러리



■ RAG 구현시 필요한 것

- 데이터
- 벡터 데이터베이스
- 프레임워크 – 랭체인

■ 랭체인 훑어보기

■ 랭체인을 사용하기 위한 환경 구성

- 아나콘다 환경 구성
- 라이브러리 설치
- API 키 발급

랭체인 사용 환경 구성 – API 키

■ OpenAI와 허깅페이스에서 제공하는 API를 이용하기 위해서는 API Key를 발급받아야 함

■ OpenAI API Key

- 발급 및 환경 변수 설정

```
import os  
os.environ["OPENAI_API_KEY"] = "발급 받은 API 사용 Key"
```

■ 허깅페이스 LLM 사용 키

- 발급 및 환경 변수 설정

```
import os  
os.environ["HUGGINGFACEHUB_API_TOKEN"] = "발급 받은 API 사용 Key"
```

■ RAG 구현시 필요한 것

- 데이터
- 벡터 데이터베이스
- 프레임워크 – 랭체인

■ 랭체인 훑어보기

■ 랭체인을 사용하기 위한 환경 구성

- 아나콘다 환경 구성
- 라이브러리 설치
- API 키 발급